La couche transport

IV. La couche transport

1. Rôles de la couche Transport
2. Protocole **TCP** (Transmission Control Protocol)
3. Protocole **UDP** (User Datagram Protocol)

La couche transport

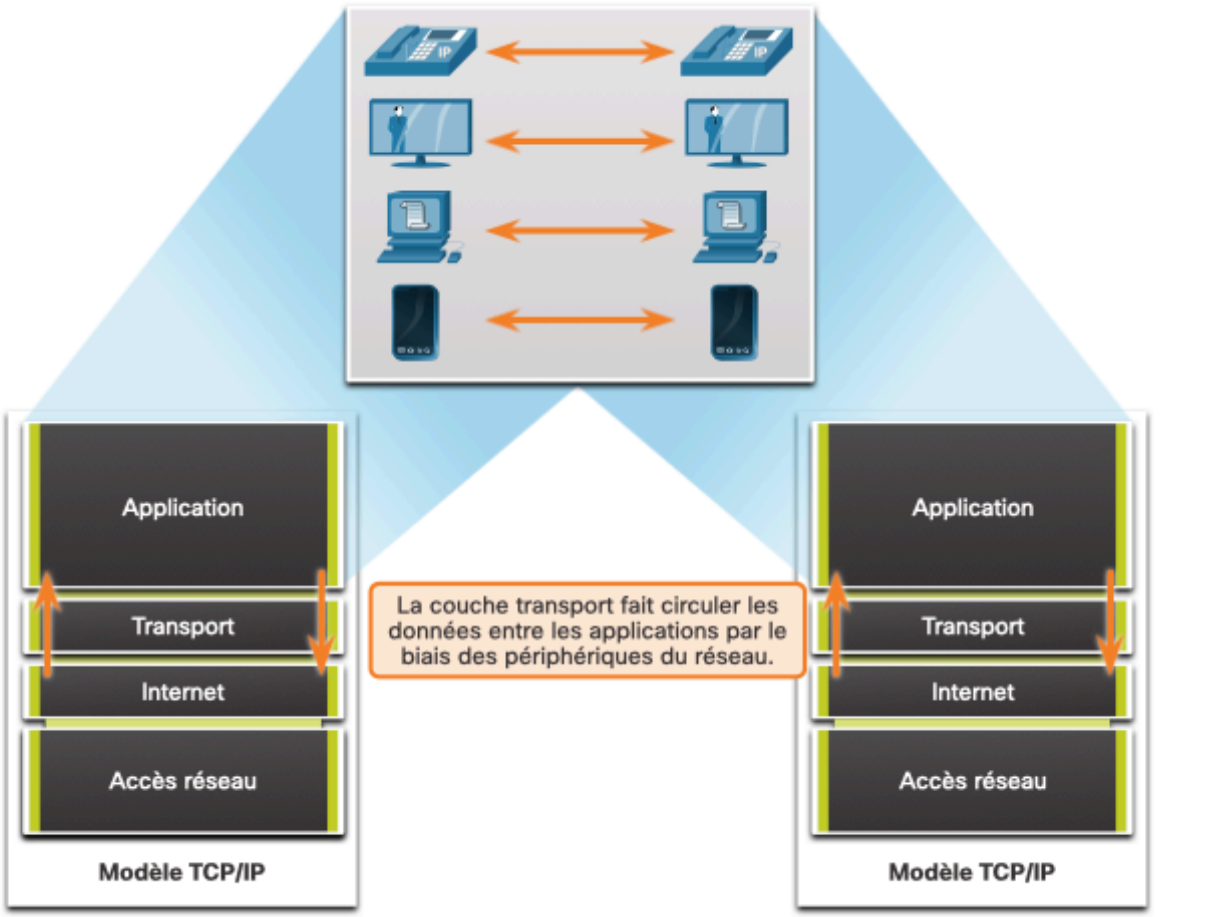
La couche application **génèrent des données** à échanger entre source et destination.

La couche de transport est responsable **des communications logiques** entre les applications exécutées sur différents hôtes.

La couche transport constitue la liaison entre la couche application et les couches inférieures.

Deux protocoles:

- **TCP** (Transmission Control Protocol)
- **UDP** (User Datagram Protocol)



Rôles de la couche Transport

La couche transport a plusieurs rôles :

a) **Segmentation des données applications en blocs** adaptés aux protocoles de transmission utilisés aux couches inférieures

b) **Identification des différentes applications** dans le cas de **communications simultanées** ==> **N° de port**

c) Définition de **différentes options de modes de transport** selon les **besoins**

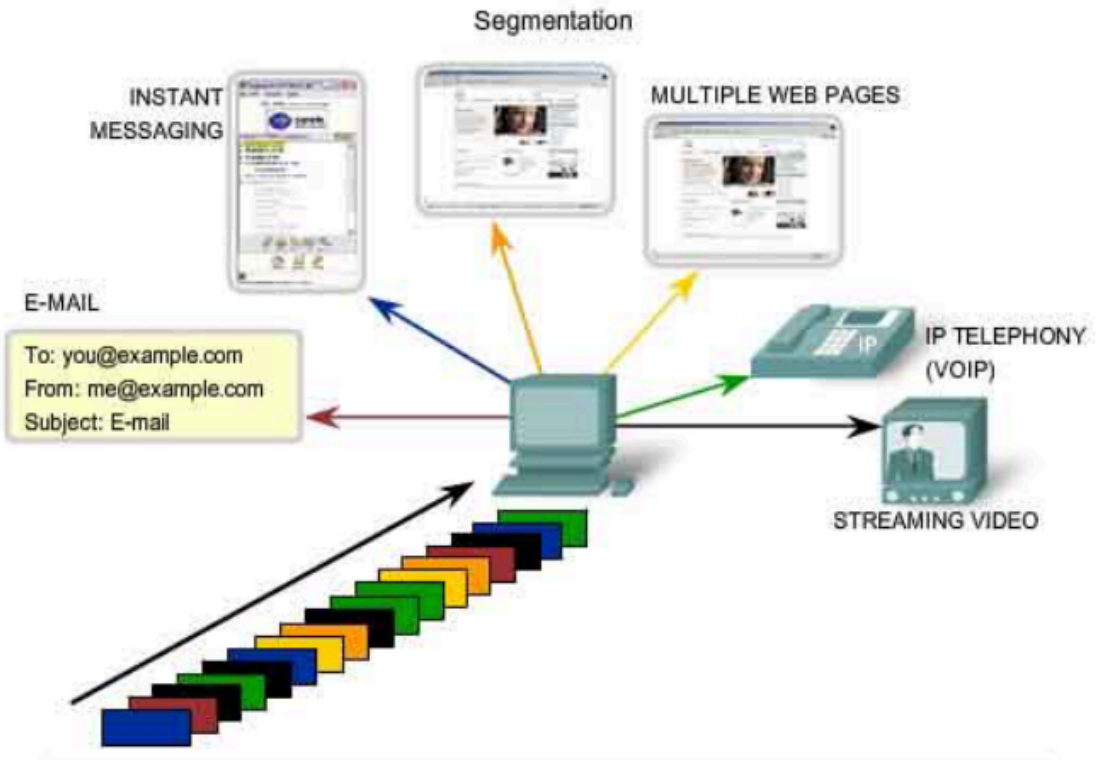
Rôles de la couche Transport

Segmentation des données en blocs :

- la couche transport **segmente** (divise) les données d'application à envoyer en plusieurs blocs de données d'une taille adéquate

- La couche transport ajoute des informations **d'en-tête** à chaque bloc (segment ou datagramme).

- La couche transport **réassemble** les blocs de données dans le bon ordre.



Rôles de la couche Transport

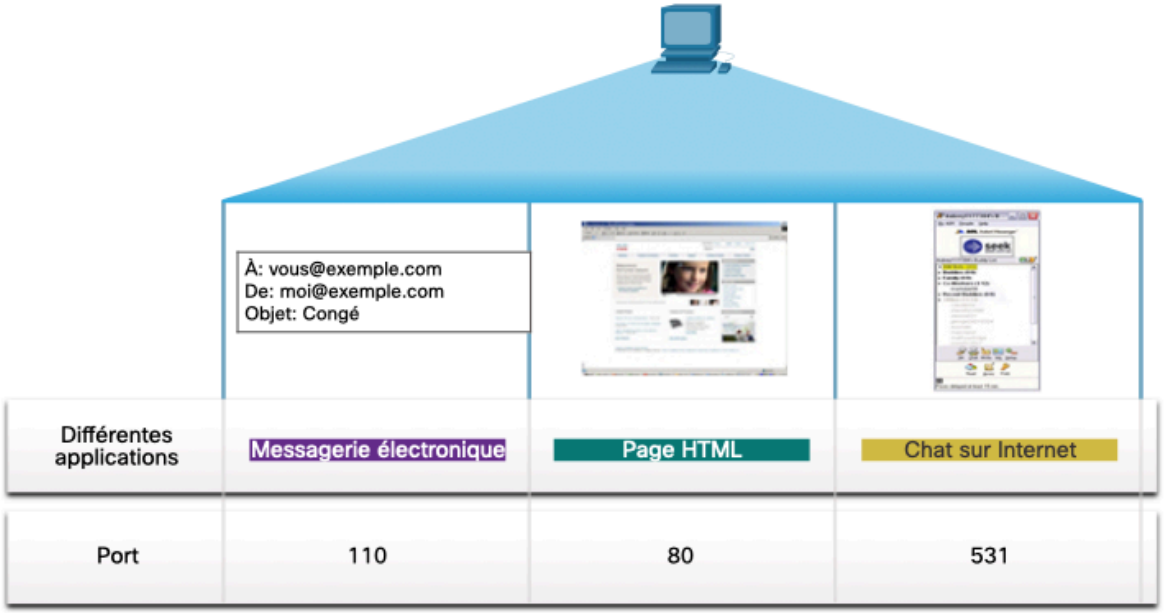
Identification des applications :

Pour transmettre les flux de données aux applications appropriées, la couche transport affecte un **identificateur unique à chaque application ou processus logiciel**, appelé **n° de port**. Ce **n° de port** est **inclus dans l'en-tête de la couche transport**.



Le numéro de port source : associé à l'application d'origine sur l'hôte local

Le numéro de port de destination : associé à l'application de destination sur l'hôte distant.



Rôles de la couche Transport

Identification des applications : L'affectation des n° de port?

- **Statique** : ==> n° de port configuré **par défaut** ou **manuellement**
  ==> **cas des processus serveur.**

- **Dynamique**: ==> n° de port est généré aléatoirement, il fait office d'**identifiant de retour** pour l'application envoyant la requête.
  ==> **cas des processus clients.**

Rôles de la couche Transport

Identification des applications : L'affectation des n° de port?

L'IANA (Internet Assigned Numbers Authority) = organisme de normalisation

➔ a divisé la gamme de numéros en 3 groupes :

Plage des numéros de ports	Groupe de ports	Pour quel type d'application?
0 à 1023	Ports **connus**	Réservés pour *les applications standards* . **Exp: HTTP :80, DNS :53, SMTP: 25**
1024 à 49151	Ports **enregistrés**	Pour les applications *non courantes* **Exp: H323: 1720** (pour l'audio, de la vidéo, et des données à travers les réseaux IP) **Exp : 1812 :le serveur RADIUS**
49152 à 65535	Ports **dynamiques et/ou privés**	*Applications clientes* lorsqu'une connexion est initiée

- *Liste des n° de ports sur* **http://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml**

ESIEE PARIS

Rôles de la couche Transport

Exemples de numéros de port connus

Numéro de port	Protocole	Application
20	TCP	FTP (File Transfer Protocol) - Données
21	TCP	FTP (File Transfer Protocol) - Contrôle
22	TCP	SSH (Secure Shell)
23	TCP	Telnet
25	TCP	Protocole SMTP
53	UDP, TCP	Service de noms de domaine (Domain Name System, DNS)
67	UDP	Serveur DHCP (Dynamic Host Configuration Protocol)
68	UDP	Protocole DHCP (Dynamic Host Configuration Protocol) (client)
69	UDP	Protocole TFTP (Trivial File Transfer Protocol)
80	TCP	Protocole HTTP (Hypertext Transfer Protocol)
110	TCP	protocole POP3 (Post Office Protocol version 3)
143	TCP	IMAP (Internet Message Access Protocol)
161	UDP	Simple Network Management Protocol (SNMP)
443	TCP	protocole HTTPS (Hypertext Transfer Protocol Secure)

Rôles de la couche Transport

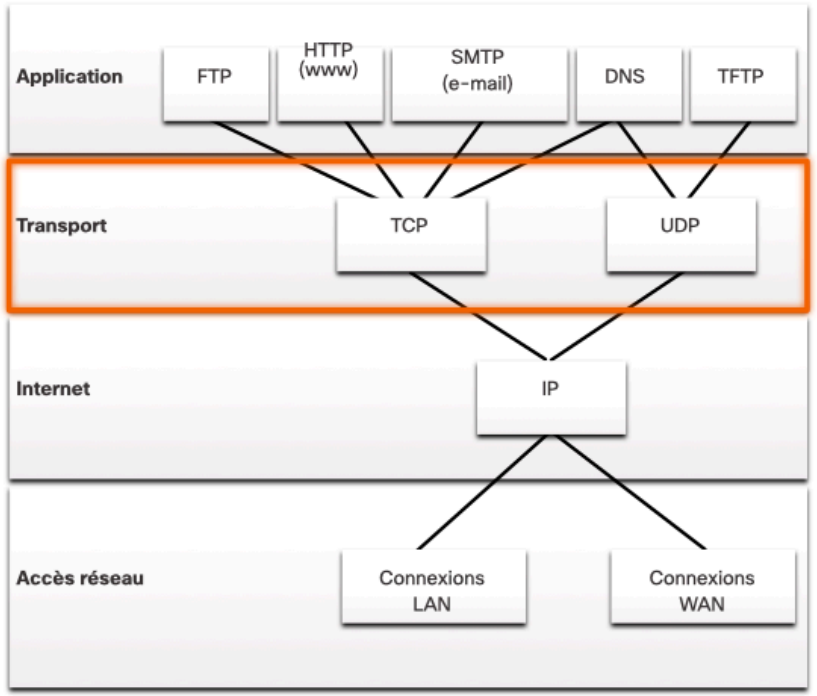
Options de transports selon les besoins

- TCP/IP fournit deux protocoles de la couche transport, **TCP et UDP,** car toute les applications **n'ont pas besoin du même degré de fiabilité et de rapidité**.

1. **Transmission Control Protocol (TCP)**
   - Assure un acheminement fiable – Toutes les données arrivent à destination
   - Utilise les accusés de réception et d'autres mécanismes pour garantir la transmission
   - Sollicite davantage le réseau, et le surcharge plus

2. **User Datagram Protocol (UDP)**
   - Fournit juste les fonctions de base pour la transmission, sans aucune garantie
   - Moins de surcharge

- **TCP ou UDP ?**
  - Compromis entre l'importance accordée à la **fiabilité** et **la charge imposée au réseau**
  - Les développeurs d'applications choisissent le protocole de transport en fonction des besoins



La couche transport

UDP vs TCP

UDP



VoIP
(IP telephony)



DNS
(Domain Name Resolution)

Propriétés de protocole requises:

- Rapide
- Faible surcharge
- Pas d'accusé de réception requis
- Pas de renvoi des données perdues
- Envoi des données à mesure de leur arrivée

TCP



SMTP/IMAP
(E-mail)



HTTP/HTTPS
(World Wide Web)

Propriétés de protocole requises:

- Fiable
- Accuse réception des données
- Renvoi des données perdues
- Envoi des données en ordre séquentiel

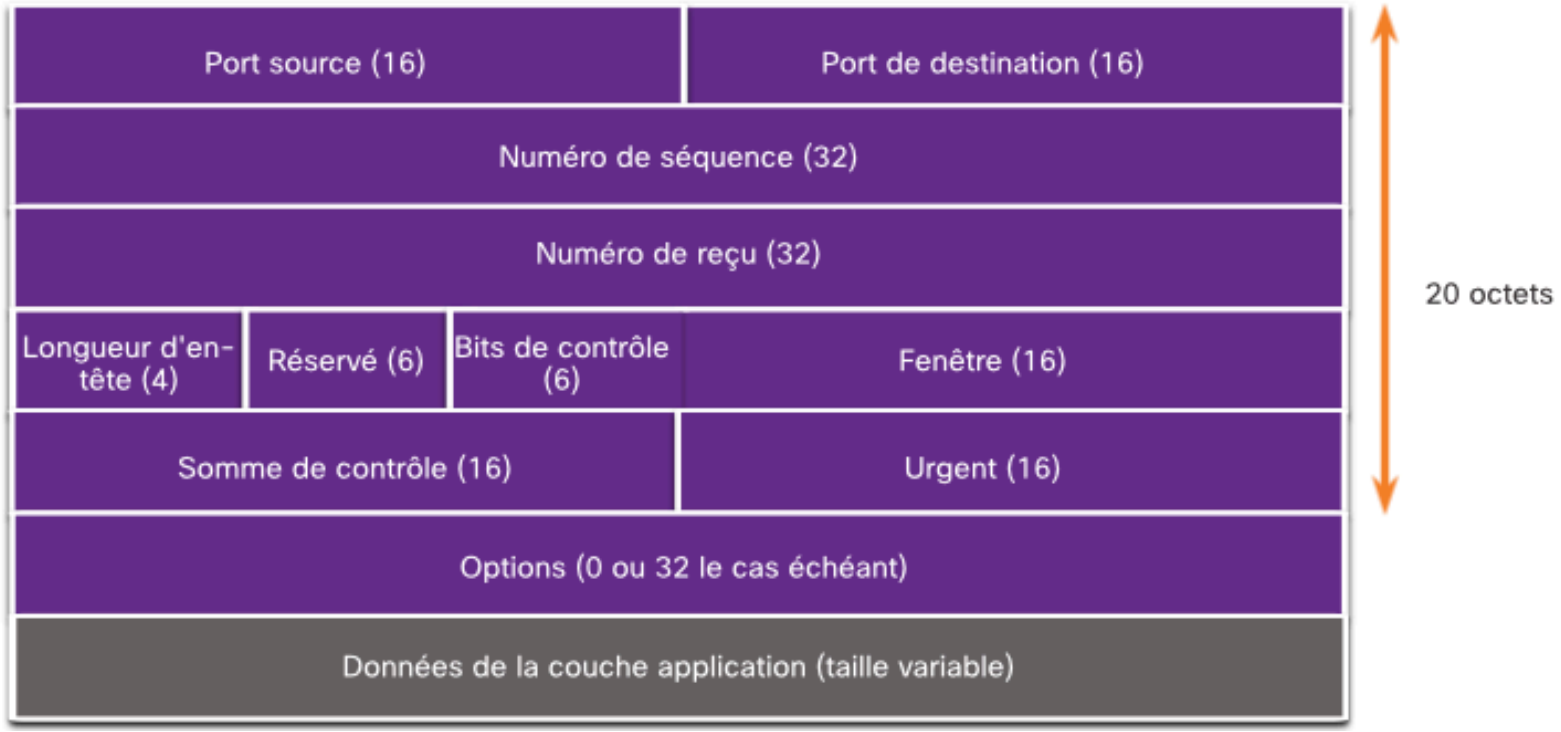
La couche transport

TCP

PDU ==> des segments

Utilisation ==>HTTP, FTP, SMTP…

En-tête ==>20 octets (8 pour UDP)



La couche transport

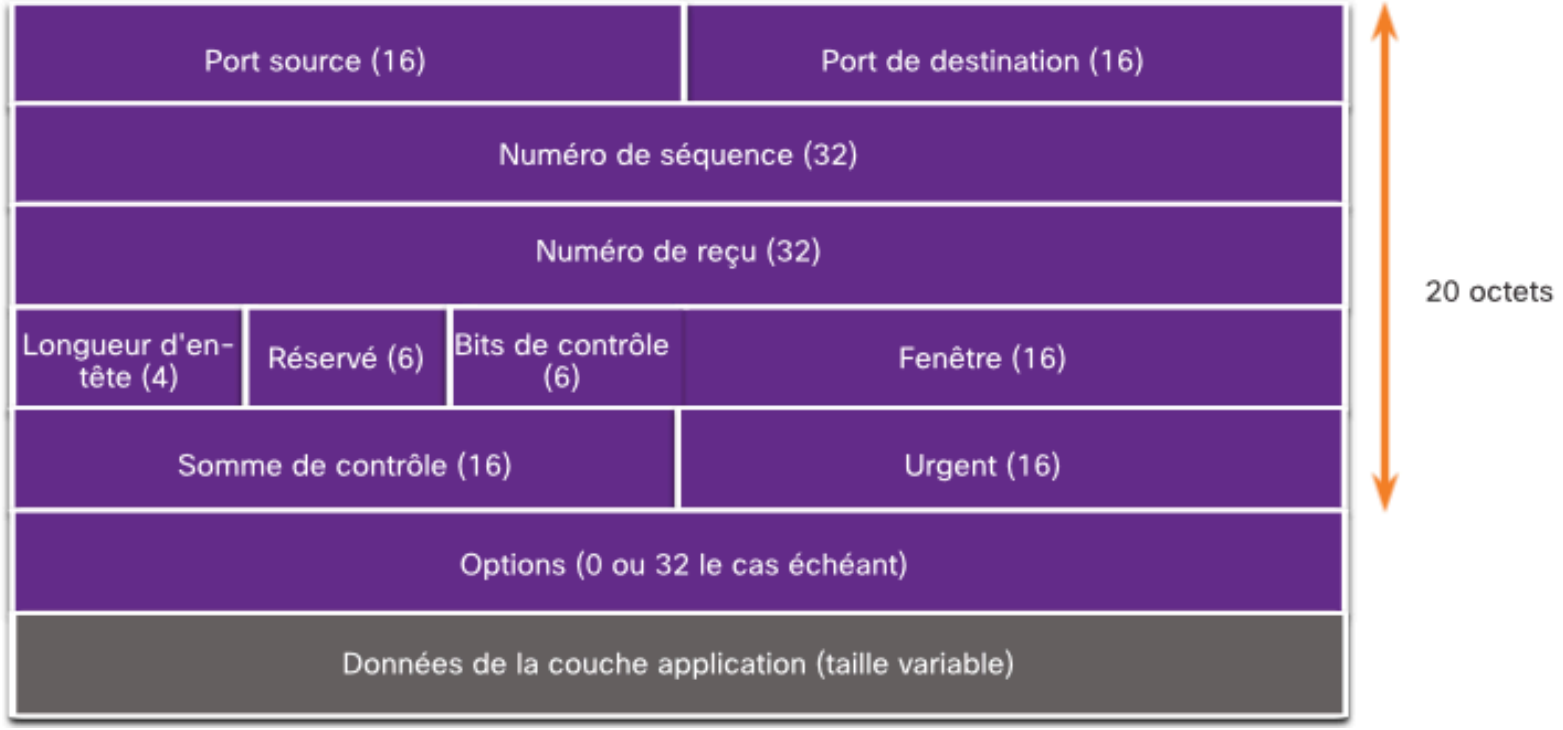
TCP

N° de port source (16 bits)

session TCP sur le périphérique ayant ouvert une connexion

N° de port de destination (16 bits) :

identifie l'application de destination sur le site distant

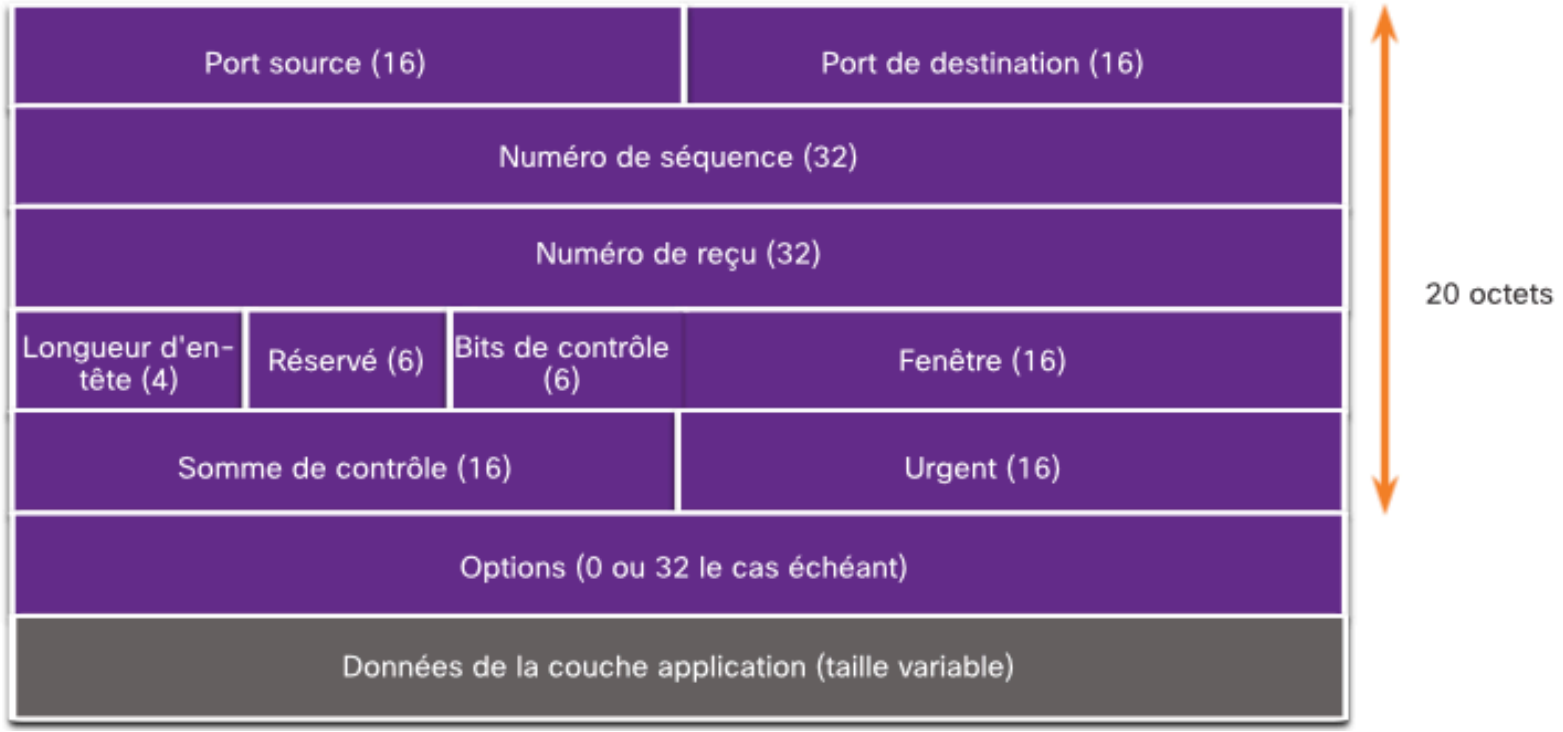


La couche transport

TCP

Numéro d'ordre (32 bits)
= Numéro de séquence
= Sequence number en anglais

Joue un rôle crucial afin que les données sont envoyées et reçues dans le même ordre



Par exemple: si un client envoie un segment avec un numéro d'ordre de 1001, la taille d'un segment est de 500 octets :

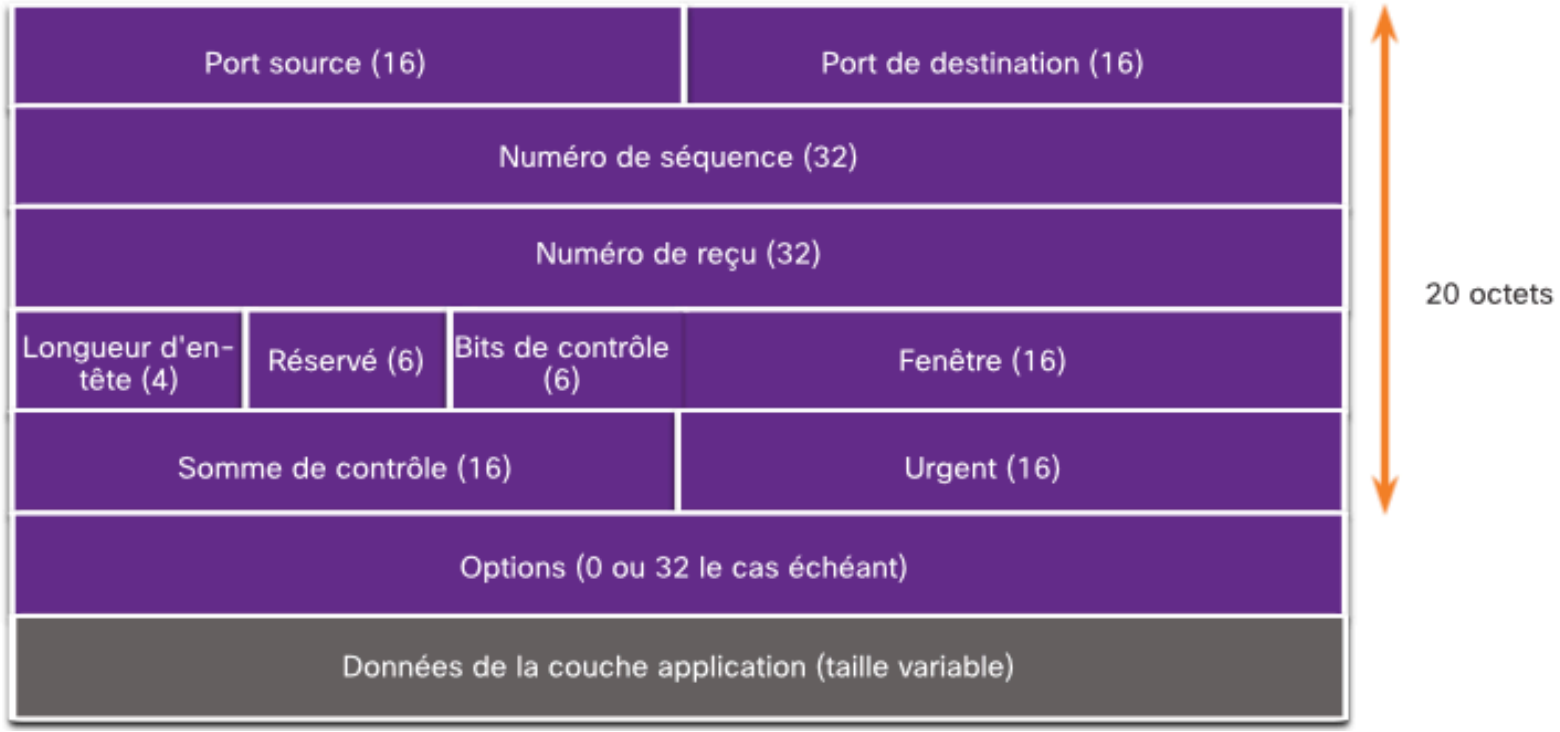
- Cela signifie qu'il a envoyé 1000 octets.
- Le prochain segment aura un numéro de séquence de 1501 ==> il correspondra au 1501e octet dans le flux.

La couche transport

TCP

Numéro de reçu (32 bits):
= Numéro d'accusé de réception
= Acknowledgment Number en anglais

Indique le **prochain numéro de séquence** que le récepteur s'attend à recevoir. Il confirme que toutes les données jusqu'à ce numéro ont été reçues avec succès.



Par exemple, si le récepteur envoie un numéro de reçu de **1001**

➔ cela signifie qu'il a correctement reçu tous les octets jusqu'au numéro de séquence 1000
➔ et qu'il attend le prochain segment commençant par l'octet 1001.

TCP

Longueur d'en-tête (4bits) :

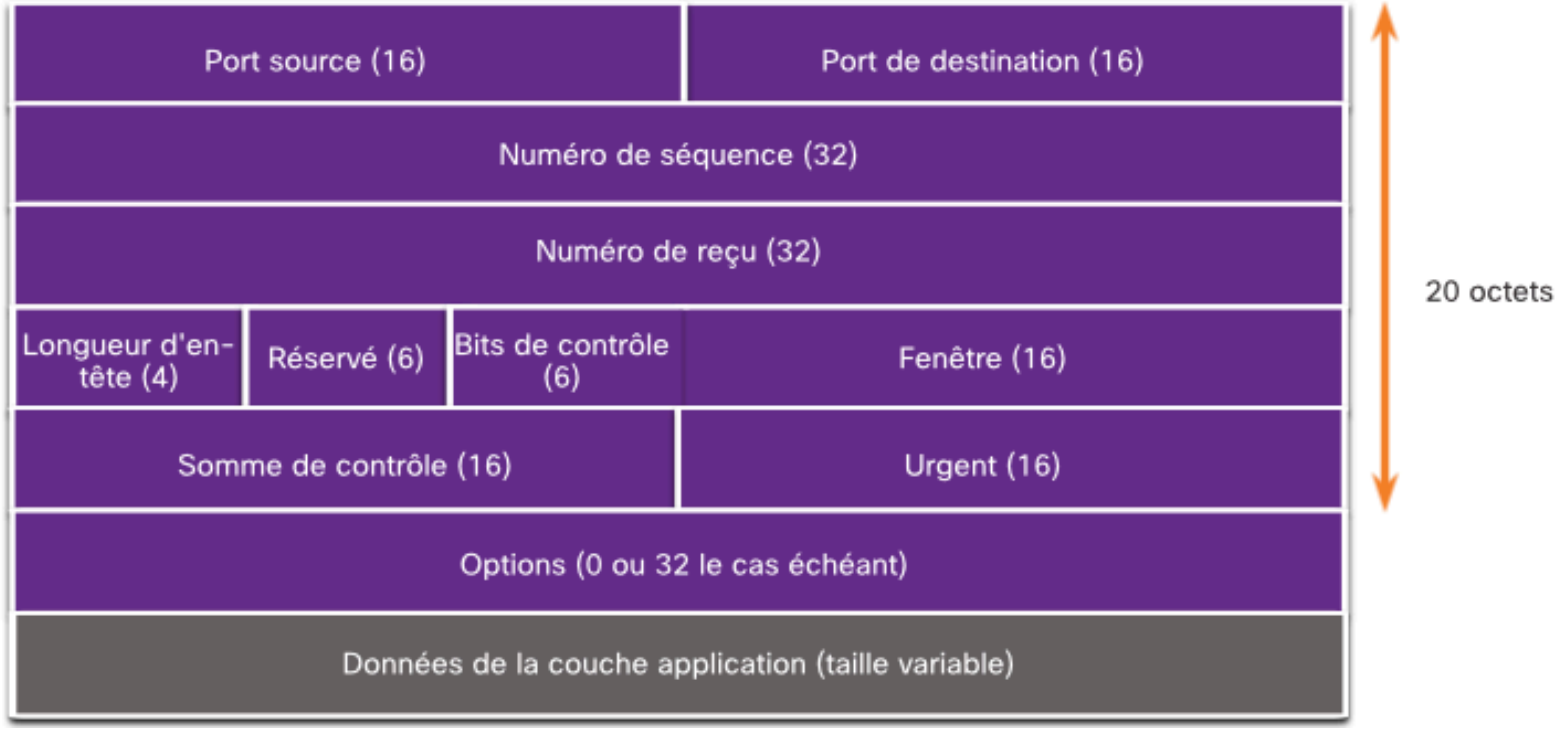
longueur de l'en-tête du segment en octets

Réservé (6 bits) :

réservé pour une utilisation future.

Bits de contrôle (6 bits):

qui inclut des codes de bits, ou des indicateurs, qui indiquent l'objectif et la fonction du segment TCP.



La couche transport

TCP

Taille de la fenêtre (16 bits) :

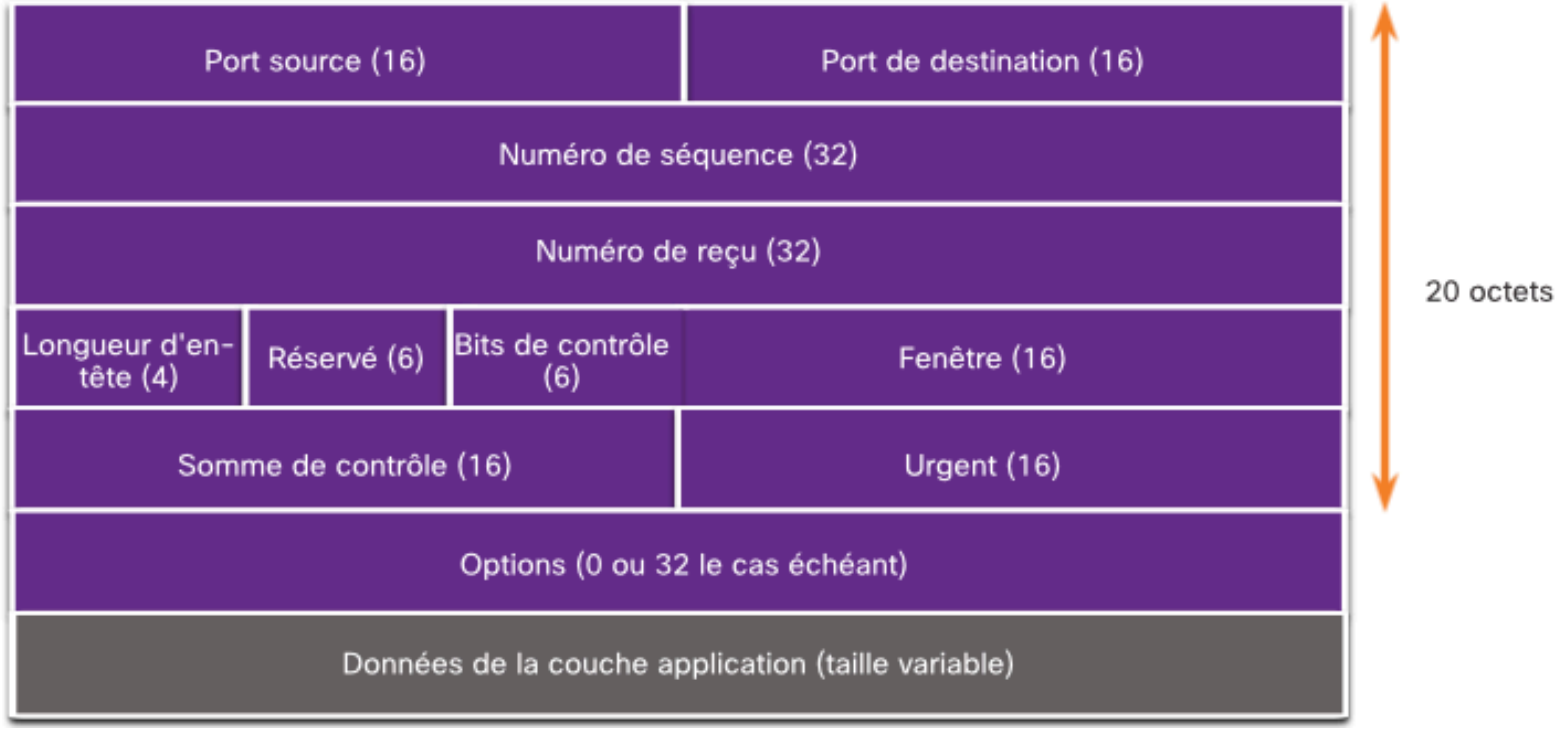
valeur de la fenêtre dynamique (combien d'octets peuvent être envoyés avant d'attendre le reçu)

Somme de contrôle TCP (16 bits) :

utilisé pour contrôler les erreurs d'en-tête et de données

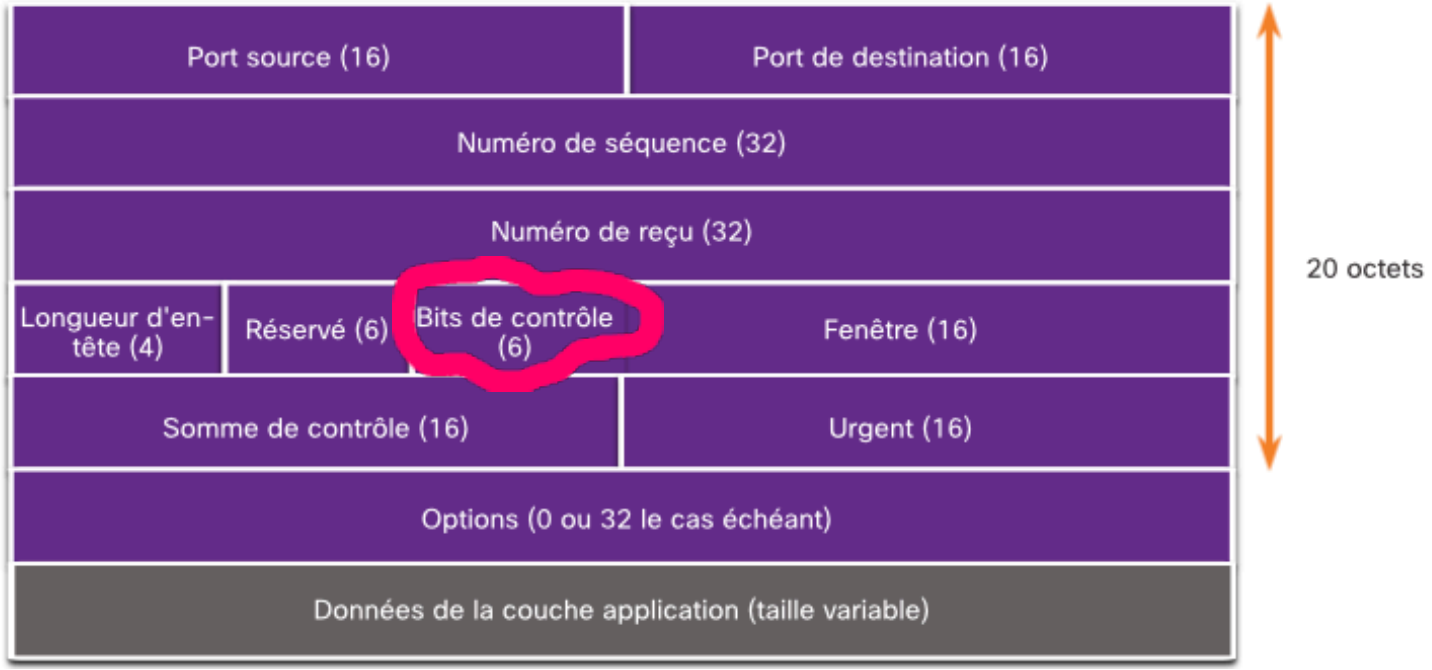
Urgent (16 bits):

utilisé pour indiquer si les données contenues sont urgentes



TCP

L'entête du segment TCP comporte 6 champs de 1 bit appelés des **indicateurs** (**flags**), ils indiquent des informations de contrôle **permettant l'initiation et la fermeture des sessions TCP**.



TCP

L'entête du segment TCP comporte 6 champs de 1 bit chacun appelés des **indicateurs** (**flags**), ils indiquent des **informations de contrôle** permettant l'initiation et la fermeture des sessions TCP.

- **URG(Urgent)** - Champ de pointeur urgent significatif (Urgent pointer field significant)
- **ACK(Acknowledgment)** - Indicateur d'accusé de réception utilisé dans l'établissement de la connexion et la fin de la session
- **PSH(Push)** - Les données doivent passer à l'application immédiatement
- **RST(Reset)** - Réinitialisation de la connexion en cas d'erreur ou de dépassement de délai
- **SYN(Synchronize)** - Synchroniser les numéros de séquence utilisés dans l'établissement de connexion
- **FIN(Finish)** - Plus de données de l'expéditeur et utilisées dans la fin de session

Ces mécanismes de connexion et de sessions permettent la fonction de **fiabilité** de TCP.

TCP

- **Etapes pour communication entre 2 hôtes avec TCP :**

1. **Ouverture de la connexion TCP =** 3 way handshake TCP (**3 étapes**)

2. **Échange de données** entre les 2 hôtes

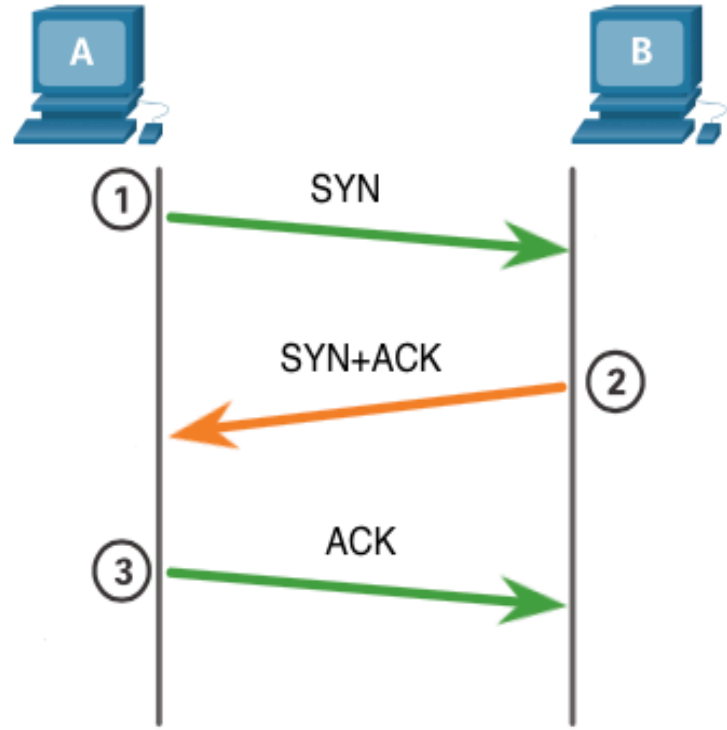
3. **Fermeture de la connexion TCP** (**4 étapes**)

TCP

Les connexions TCP sont établies via un échange appelé "**Three-way handshake**".

Si A est le client et B le serveur, la négociation se déroule comme suit:

1) A envoie à B un paquet avec le bit **SYN** activé (un paquet SYN)
2) B répond avec un paquet **SYN**, le bit **ACK** est activé également (un paquet SYN+ACK)
3) A répond à B par un paquet avec le bit **ACK** activé (un paquet ACK)



TCP Three-way handshake

TCP

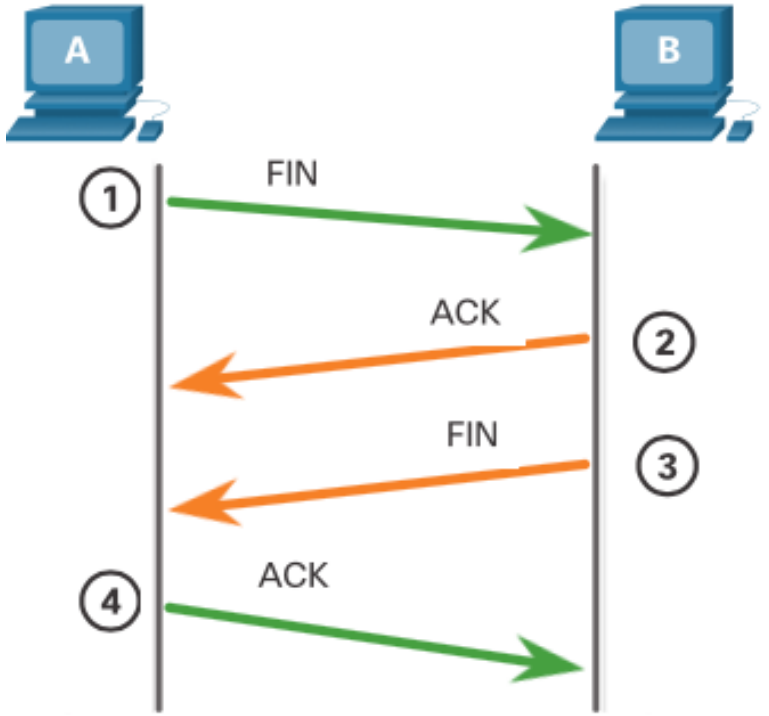
Pour fermer la connexion, un échange impliquant des paquets **FIN** peut avoir lieu :

1) A envoie à B un paquet avec le bit **FIN** activé (un paquet FIN), annonçant la fermeture de la session TCP.

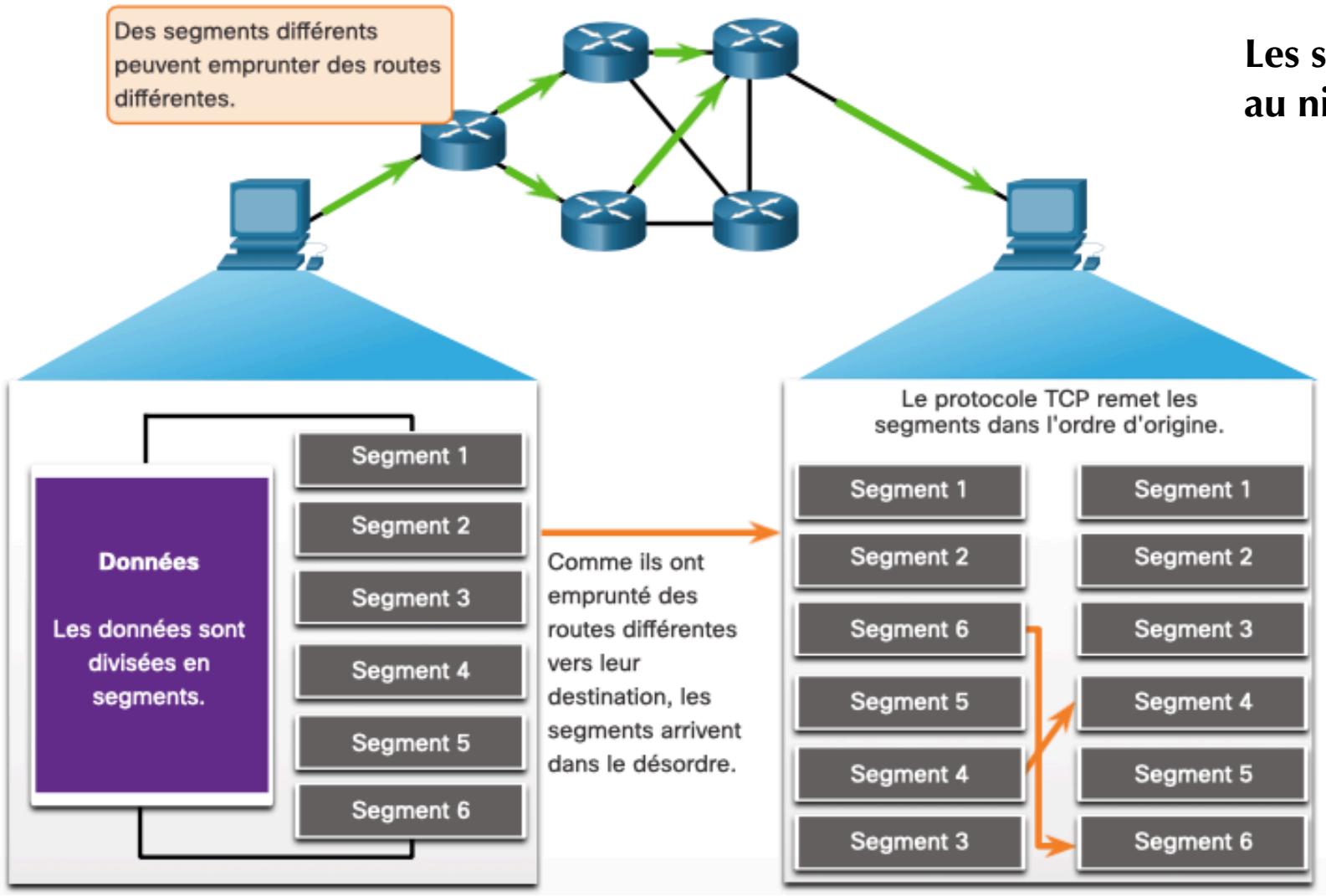
2) B envoie un segment ACK pour informer de la bonne réception du segment FIN afin de fermer la session.

3) B envoie un segment FIN au client A pour mettre fin à la session .

4) A envoie à B un **ACK** pour accuser la réception du FIN.



TCP



Les segments TCP sont réorganisés au niveau de la destination

Deux champs de l'entête du protocole TCP sont utilisé

numéros d'ordre et numéros de reçu

La couche transport

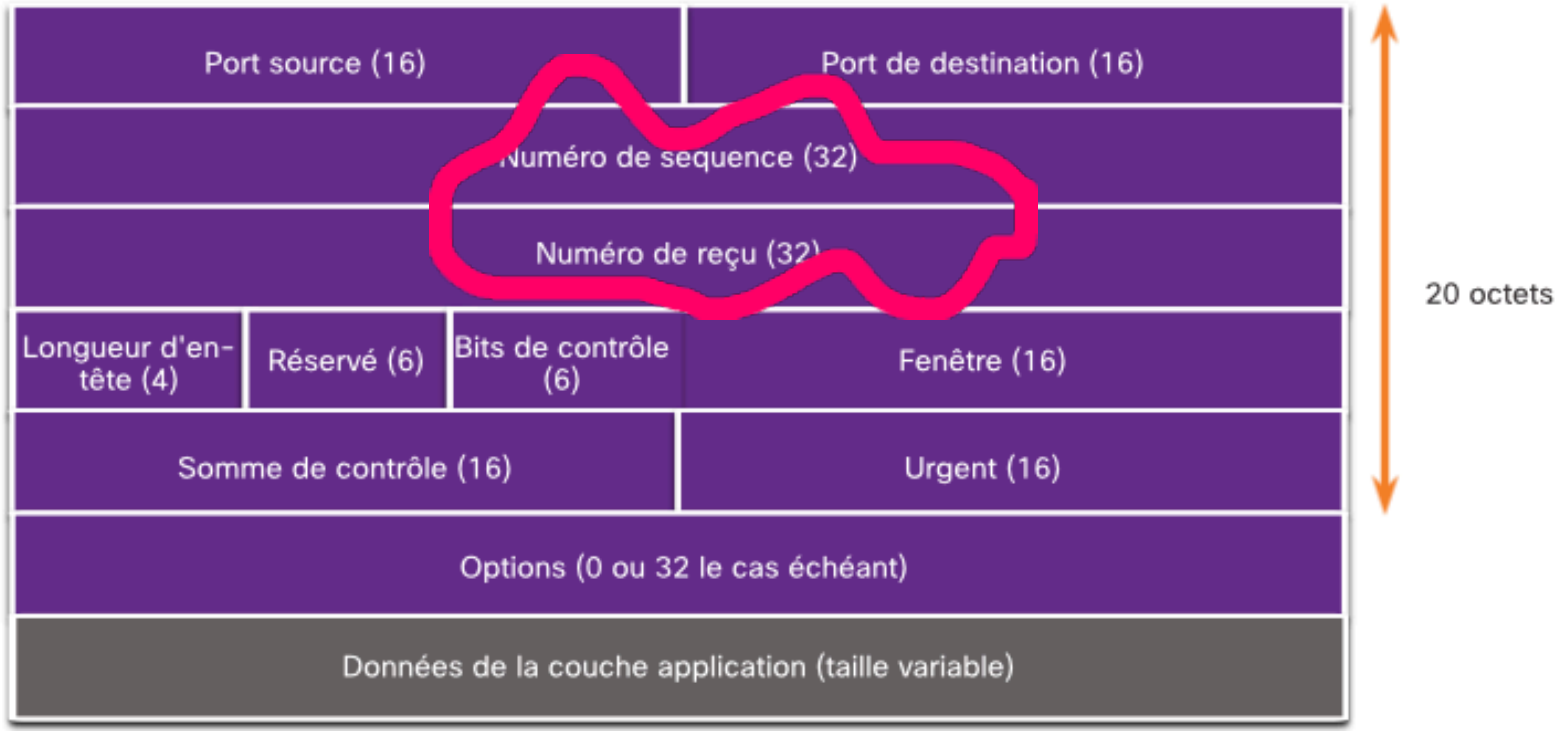
TCP

- **L'ordonnancement, comment?**

N° d'ordre (32 bits) :
Utilisé à des fins de réassemblage de données

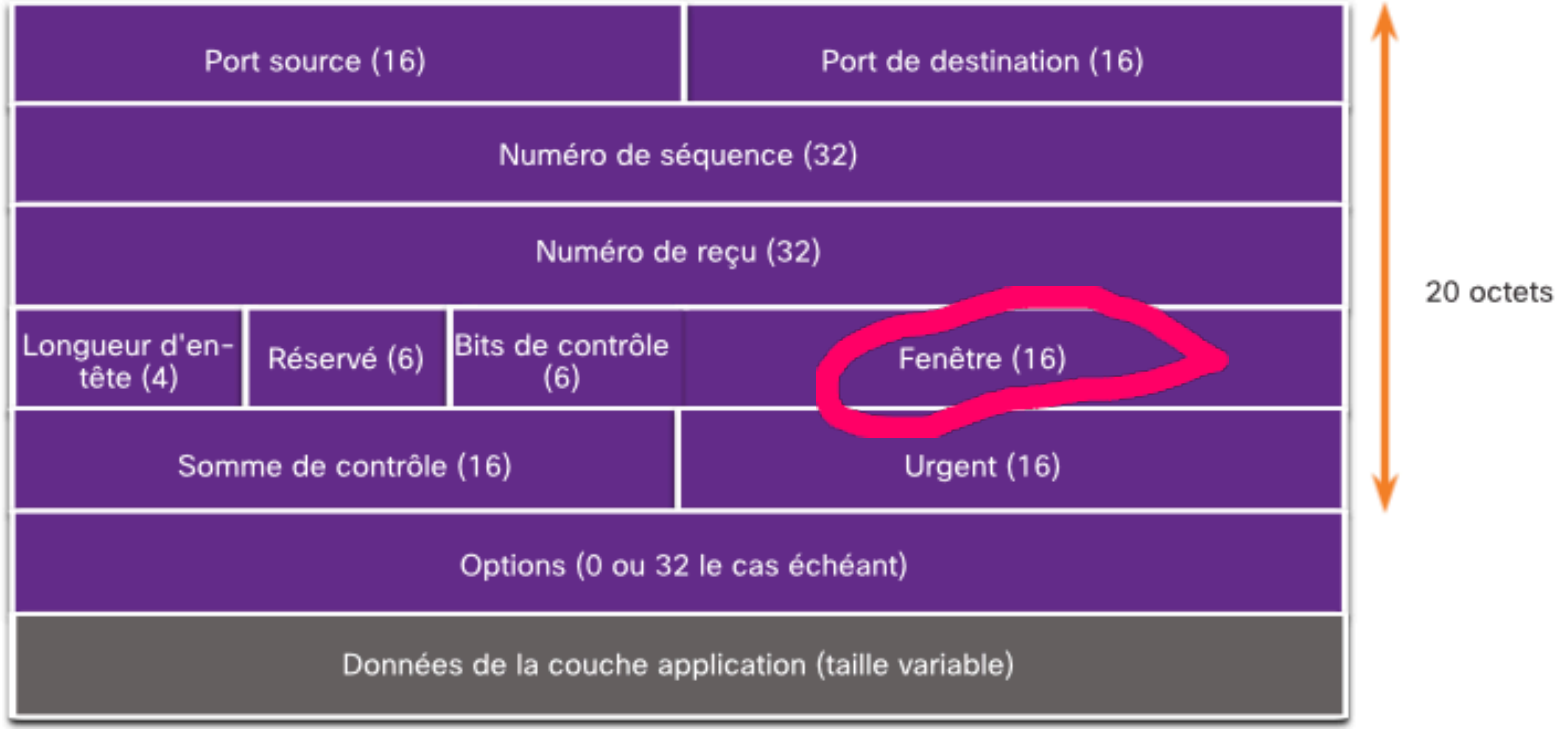
le **n° d'ordre est incrémenté du nombre d'octets ayant été transmis**. Cela permet d'identifier chaque segment et d'en accuser réception individuellement.

N° de reçu (32 bits):
utilisé pour indiquer que les données ont été reçues et que le prochain octet est attendu de la source.



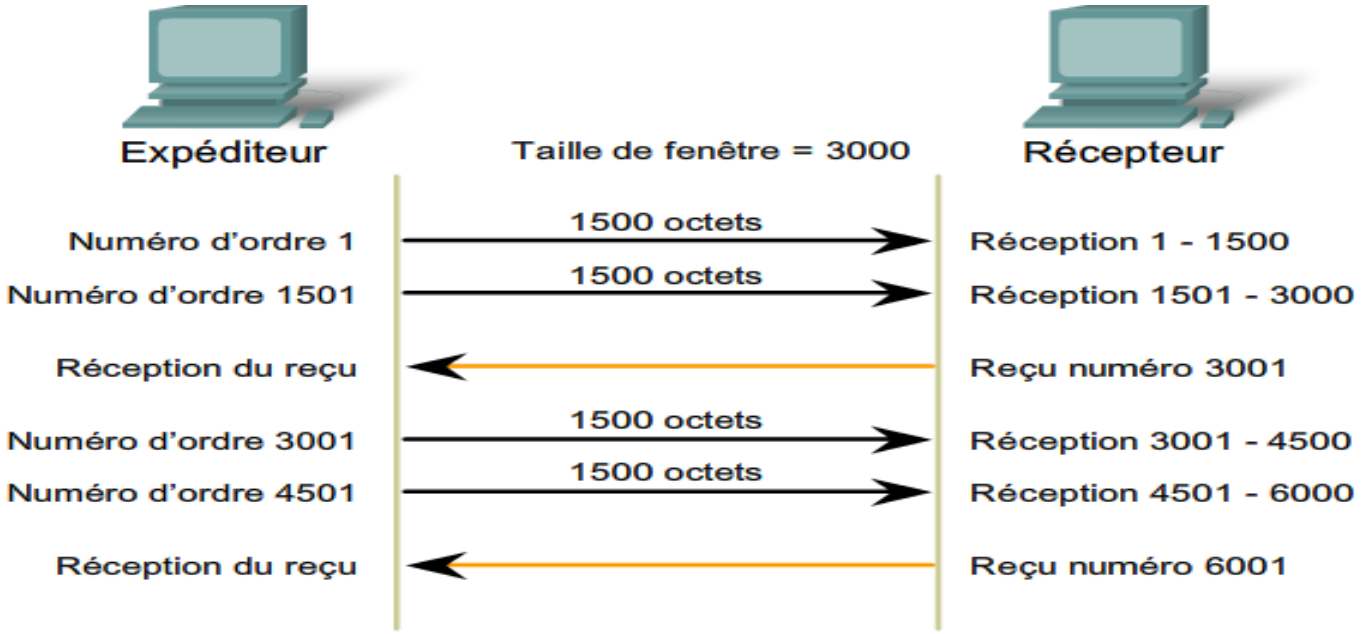
TCP

- Taille de la fenêtre ?



TCP

Contrôle de flux TCP – Taille de fenêtre et accusé de réception

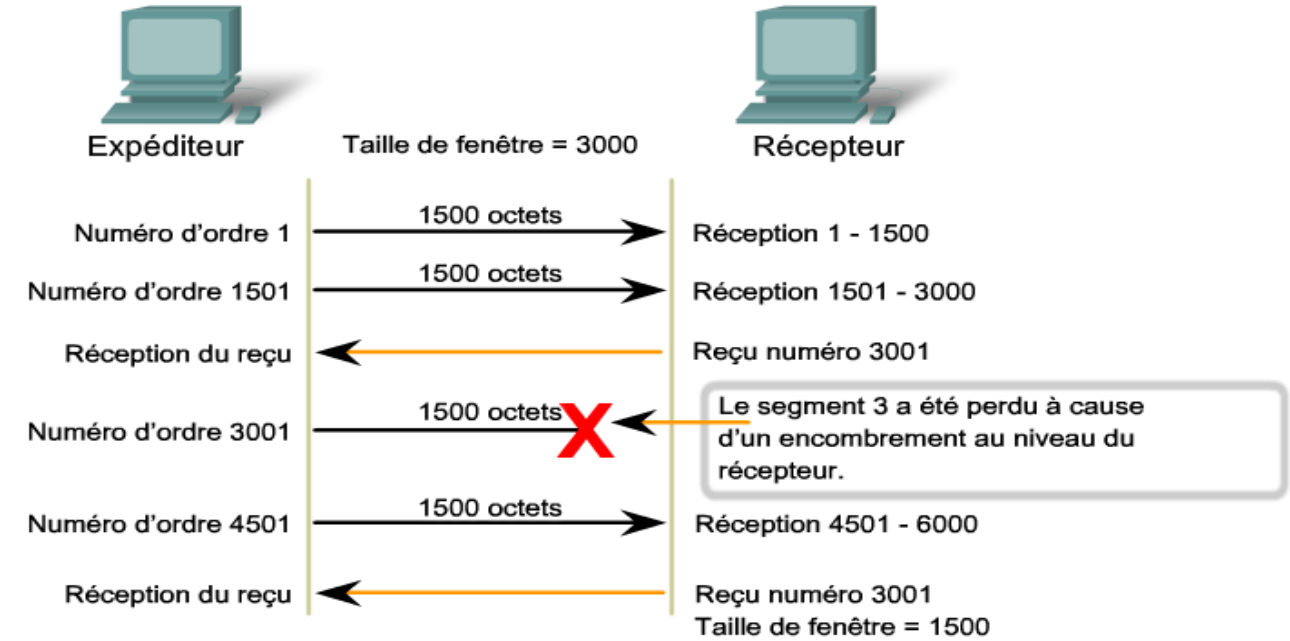


La taille de fenêtre détermine le nombre d'octets envoyés avant qu'un reçu ne soit attendu.
Le numéro du reçu est le même que le numéro du prochain octet attendu.

TCP

- Quand la perte de données est trop fréquente, le protocole TCP peut **réduire la taille de fenêtre** afin d'imposer l'envoi plus fréquent de reçus.

La Taille de la fenêtre est dynamique



Si des segments sont perdus du fait d'un encombrement, le récepteur enverra un reçu pour le dernier segment séquentiel reçu et répondra en utilisant une taille de fenêtre réduite.

UDP

- **UDP = User Datagram Protocol**
- UDP est un protocole **simple** et **sans connexion**
- **Peu de surcharge** pour l'acheminement des données
- PDU UDP appelés **datagrammes**
- En-tête UDP réduit à **8 octets**
- Utilisé par services DNS, DHCP, TFTP, …, routage, et les applications **temps réel** …
- Les datagrammes envoyés ***«au mieux»,*** sans garantie: pas de mécanismes de retransmission, ni séquençage, ni contrôle de flux.



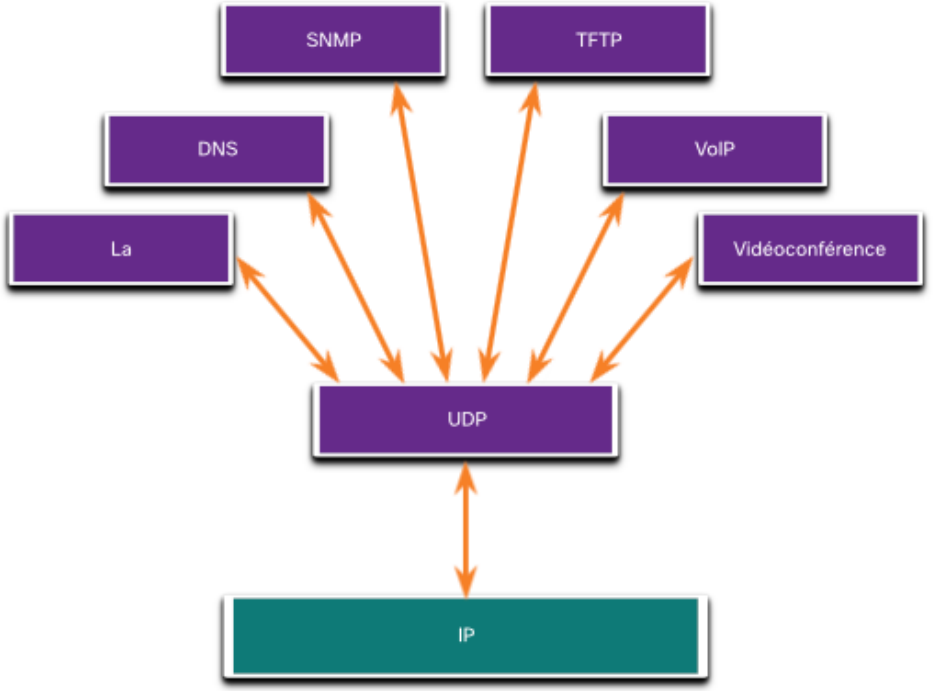
UDP

Applications utilisant l'UDP

- **Applications vidéo et multimédia en direct** :
  - Tolèrent une certaine perte de données
  - Tolèrent peu ou pas de délai
  - Exemples : La voix sur IP et le streaming vidéo

- **Applications simples de demande et de réponse :**
  - Utilisent des transactions simples où un hôte envoie une demande et peut ou ne peut pas recevoir une réponse.
  - Exemples : DNS, DHCP.

- **Applications qui gèrent elles-mêmes la fiabilité :**
  - L'application elle-même gère le contrôle de flux, la détection des erreurs, les accusés de réception…
  - Exemples : SNMP, TFTP



UDP

Avantages du protocole UDP

- **Rapidité :** pas de connexion à établir, pas d'accusé de réception à attendre, ni de retransmission

  UDP est plus rapide que TCP.
- **Faible surcharge :** L'en-tête léger de 8 octets permet d'envoyer plus de données utilisateur dans un paquet unique.
- **Flexible pour les applications temps réel :** UDP est préféré dans des applications où les délais sont inacceptables, même si des pertes de données peuvent se produire.

Inconvénients du protocole UDP

- **Aucune fiabilité** : Il n'y a aucune garantie que les paquets arriveront à destination.
- **Pas de contrôle de flux** : UDP ne gère pas la vitesse d'envoi des données, ce qui peut entraîner une surcharge du réseau en cas d'envoi massif.
- **Pas d'intégrité des données** : Si les paquets arrivent corrompus, ils sont simplement ignorés, ce qui peut affecter la qualité des données reçues.